

What a Product Costs

The same multiplications and additions, counted, and then measured in operations per second: the numbers differ by factors of thousands.

Count the operations

- The innermost line of the loops: one multiplication, one addition
- `total += W[j, c] * X[i, c]` runs once for every point, neuron and coordinate:

$$n \times K \times m$$

- **Predict:** n doubled? n, m and K all doubled?
- Measured on random numbers: n = 400, m = 30, K = 20, then doubled

n	m	K	n x K x m	seconds	ratio to the first	nanoseconds per pass
400	30	20	240,000	0.040	1.0	166
800	30	20	480,000	0.080	2.0	167
800	60	40	1,920,000	0.322	8.1	168

$O(nKm)$

- time $\approx$ (time of one pass) $\times n \times K \times m$
- **$O(nKm)$** : how the count grows; the constant is dropped
- $n = m = K$: the count is n^3 , so $\times 8$ for each doubling
- That is the factor of Act 2, from 2,048 to 4,096
- A transpose and a reshape cost nothing: no number moves

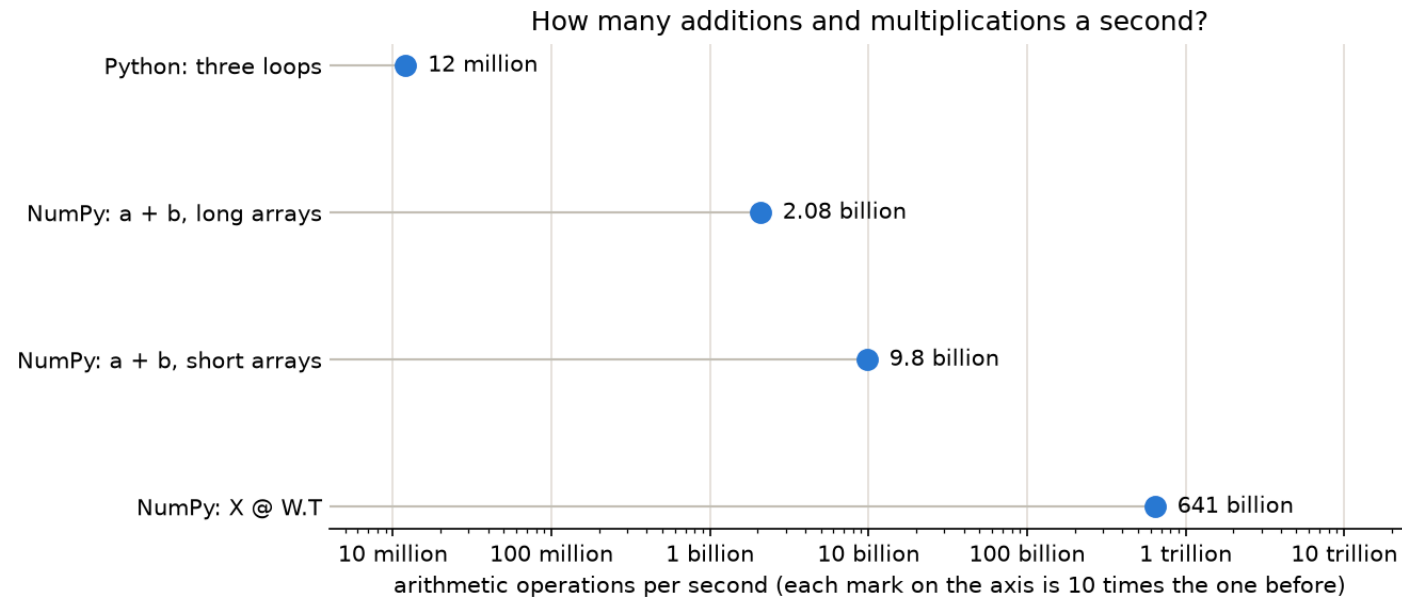
The page **What matrix operations cost**: ikoutis.github.io/cs301-labs/unit-2/pages/matrix-costs.html

The same operations, in other seconds

- The count says how the time **grows**, not how long one operation takes
- Measured: **arithmetic operations per second**, five ways
- Python, three loops · NumPy `a + b` on long arrays · on short arrays, the same additions
- NumPy `X @ W.T` at size 2,048 · **the GPU** at size 4,096, from Act 2

way of computing	operations per second	ratio to the slowest
Python: three loops	12 million	1
NumPy: <code>a + b</code> , long arrays	2.08 billion	173
NumPy: <code>a + b</code> , short arrays	9.8 billion	813
NumPy: <code>X @ W.T</code>	641 billion	53,187

How many additions and multiplications a second?



- Each mark on the axis: ten times the one before
- Four ways on the machine of these slides; **the fifth mark, the GPU, is yours**
- Python against NumPy: the interpreter's work around each operation

Long arrays against short arrays: memory-bound

- Two rows: **the same additions, by the same code**
- Only the place of the numbers differs
- **Cache**: a small, fast memory in the processor; the short arrays stay in it
- Long arrays: fetched from the main memory, used once, sent back
- **Memory-bound**: the memory sets the speed, not the arithmetic

The page **CPU and GPU**, parts 3 and 4

The product: compute-bound

- In $X @ W.T$ each entry of X meets the weights of **every** neuron
- NumPy works on blocks that fit the cache; a number is used many times
- **Compute-bound**: the time goes into arithmetic; every core helps
- The GPU: thousands of small cores on the same product
- Pays for large compute-bound work; not for small products
- $O()$ is the first estimate; the chart shows what it leaves out

Predict the time of a layer you cannot try

$$\text{time} \approx \frac{\text{number of operations}}{\text{operations per second}}$$

- $K = 1,000$ neurons on $n = 60,000$ grey images of 28×28 pixels, $m = 784$
- $2 \times n \times K \times m$ operations

$n = 60,000$, $m = 784$, $K = 1,000$:	94,080,000,000 operations (94.1 billion)
way of computing	predicted time
Python: three loops	2.17 hours
NumPy: $X @ W.T$	147 milliseconds

- The same computation, the same $O()$, three times

Now in Colab: Act 3

- **Act 3** of the class notebook: predict the two factors, then run the measurements
- The GPU's rate comes from your Act 2 measurement, in the same notebook
- Change the three sizes of the layer in the last cell; read the predicted times
- **Post**, as a reply to your first post: the line, the chart with one sentence, the two most surprising rows
- One member posts; the names of all the members first

What we saw today

	one point x	n points X
one neuron: w, b	$w @ x + b$	$X @ w + b$
K neurons: $W (K, m), b (K,)$	$W @ x + b$	$X @ W.T + b$

- The same line in NumPy and in PyTorch, on the CPU and on a GPU
- A layer of neurons: a matrix with one neuron in each row
- $O(nKm)$ operations; what a second is worth depends on the program, the memory and the processor
- **After class:** the full notebooks, and Review Quiz 11